Apostila do Laboratório de Linguagens de Programação

Módulo C++: Programação Orientada a Objetos

Pedro Henrique Ribeiro Dias

15 de outubro de 2026

Sumário

1 Relatório 6: Programação Orientada a Objetos em C++ 3
1.1 Fundamentos Teóricos e Pilares da POO . . . 3
1.1.1 A Mudança de Paradigma: Da Programação Estruturada à Orientação a Objetos . . . 3
1.1.2 Os Blocos Fundamentais: Classes, Objetos, Estado e Comportamento . . . 3
1.1.3 Os Quatro Pilares da Orientação a Objetos . . . 4
1.2 Documentação Sintática das Estruturas de POO . . . 6
1.2.1 Declaração de Classe e Modificadores de Acesso . . . 6
1.2.2 Construtores e Lista de Inicialização . . . 6
1.2.3 Herança Pública . . . 6
1.2.4 Polimorfismo Dinâmico: `virtual` e `override` . . . 6
1.2.5 Destrutor Virtual (`virtual ~Destrutor()`) . . . 7
1.2.6 Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória . . . 7
1.3 Código de Referência Completo . . . 7
1.4 Exercícios Práticos do Módulo . . . 9

2 Relatório 7: Programação Orientada a Objetos em C# 10
2.1 Fundamentos Teóricos e Pilares da POO . . . 10
2.2 Documentação Sintática das Estruturas de POO . . . 10
2.2.1 Estrutura do Programa e Saída no Console . . . 11
2.2.2 Declaração de Classe e Propriedades . . . 11
2.2.3 Construtores e a Palavra-chave `this` . . . 11
2.2.4 Herança e Repasse ao Construtor Base (`: base(...)`) . . . 12
2.2.5 Polimorfismo: `virtual` e `override` . . . 12
2.2.6 Reaproveitando o Comportamento do Pai: `base.Metodo()` . . . 12
2.2.7 Composição e Agregação . . . 13
2.2.8 Coleções Genéricas: `List<T>` e `foreach` . . . 13
2.3 Código de Referência Completo . . . 13
2.4 Exercícios Práticos do Módulo . . . 17

3 Relatório 8: Programação Orientada a Objetos em Python 19
3.1 Fundamentos Teóricos e Pilares da POO . . . 19
3.1.1 Interfaces: o Contrato entre Classes . . . 20
3.1.2 Relacionamentos entre Classes . . . 20
3.2 Documentação Sintática das Estruturas de POO . . . 20
3.2.1 Estrutura do Programa e Saída no Console . . . 20
3.2.2 Declaração de Classe, `__init__` e `self` . . . 21
3.2.3 Encapsulamento por Convenção: `_` e `__` . . . 21
3.2.4 Getters e Setters . . . 22
3.2.5 Herança e `super().__init__()` . . . 22
3.2.6 Sobrescrita e `super().metodo()` . . . 22

3.2.7 Interfaces com `ABC` e `@abstractmethod` . . . 22
3.2.8 Composição e Agregação . . . 23
3.2.9 Listas, `for` e `isinstance` . . . 23
3.3 Código de Referência Completo . . . 24
3.4 Exercícios Práticos do Módulo . . . 28

4 Relatório 9: Programação Orientada a Objetos em Ruby 31
4.1 Fundamentos Teóricos e Pilares da POO . . . 31
4.1.1 Módulos: Contratos e Mixins . . . 32
4.1.2 Relacionamentos entre Classes . . . 32
4.2 Documentação Sintática das Estruturas de POO . . . 32
4.2.1 Estrutura do Programa e Saída no Console . . . 33
4.2.2 Declaração de Classe, `initialize` e Atributos com `@` . . . 33
4.2.3 Encapsulamento: `attr_reader`, `attr_writer` e `attr_accessor` . . . 34
4.2.4 Getters e Setters Manuais . . . 34
4.2.5 Herança e `super(...)` no `initialize` . . . 35
4.2.6 Sobrescrita e `super` em Métodos . . . 36
4.2.7 Módulos como Contrato: `module`, `include` e `NotImplementedError` . . . 36
4.2.8 Módulos com Implementação Pronta (Mixins) . . . 37
4.2.9 Composição e Agregação . . . 38
4.2.10 Arrays: Criação, `<<` e `.each` . . . 38
4.2.11 Transformar e Filtrar: `.map`, `.select` e `respond_to?` . . . 38
4.3 Código de Referência Completo . . . 39
4.4 Exercícios Práticos do Módulo . . . 43

Capítulo 1

Relatório 6: Programação Orientada a Objetos em C++

1.1 Fundamentos Teóricos e Pilares da POO

1.1.1 A Mudança de Paradigma: Da Programação Estruturada à Orientação a Objetos

Para compreender a Programação Orientada a Objetos (POO), é fundamental entender a evolução na forma de estruturar o raciocínio computacional. Na programação estruturada ou procedural, o código é organizado como uma sequência linear ou hierárquica de procedimentos e rotinas que operam sobre dados armazenados em estruturas separadas. Nesse modelo, existe uma separação clara entre "quem executa a ação"(a função) e "quem sofre a ação"(os dados).

Conforme os sistemas de software crescem em escala e complexidade, essa separação gera desafios significativos de manutenção. Alterar a estrutura de um dado pode exigir a modificação de dezenas de funções espalhadas pelo código, aumentando o risco de efeitos colaterais indesejados.

A POO propõe uma mudança fundamental de perspectiva: em vez de tratar dados e funções como entidades separadas, ela os unifica em estruturas coesas e autônomas. O foco da modelagem deixa de ser a sequência de passos executados pelo computador e passa a ser as **entidades do domínio do problema** e as interações entre elas.

1.1.2 Os Blocos Fundamentais: Classes, Objetos, Estado e Comportamento

Antes de analisar os pilares da POO, é necessário dominar os quatro conceitos estruturais que sustentam esse paradigma:

- **Classe (O Modelo Conceitual)**: Uma classe é uma especificação abstrata, funcionando como um projeto ou molde. Ela não ocupa espaço operacional como uma entidade ativa em memória; em vez disso, define a estrutura lógica, as propriedades que os elementos desse tipo possuirão e as ações que poderão executar. Classes podem abstrair conceitos do mundo real para o código, por exemplo, podemos criar uma classe Animal, Pessoa, Carro, ContaBancária e muito mais, portanto não se prendam ao modelar classes.

- **Objeto (A Entidade Concreta)**: Um objeto é a concretização prática de uma classe — processo conhecido como *instanciação*. Se a classe é a planta arquitetônica de uma casa, o objeto é a casa física efetivamente construída na memória do computador durante a execução do programa. Um programa em POO consiste em um conjunto de objetos interagindo por meio do envio de mensagens. Continuando o exemplo dado no final da classe, a instância de um Animal poderia ser um cachorro real, por exemplo o Scooby-doo (não confundam estes conceitos, poderíamos criar uma classe Cachorro, uma abstração

deste animal para o código, e o objeto criado, a instância dela seria o cachorro Scooby-doo!)

- **Atributos e Estado**: Os atributos são as variáveis internas pertencentes a um objeto que armazenam suas características ou dados. O conjunto de valores armazenados nesses atributos em um determinado instante do tempo define o **estado** do objeto. O estado de um objeto é dinâmico e pode se alterar ao longo do ciclo de vida da aplicação. Por exemplo, mantendo a linha de Animais, estes podem ter espécie, nome e diversas outras características expressas em seus atributos.

- **Métodos e Comportamento**: Os métodos são as rotinas ou funções associadas a uma classe que operam sobre os seus atributos. Eles representam o **comportamento** do objeto, definindo as ações que ele pode realizar, as transformações que pode aplicar ao seu próprio estado ou as informações que pode fornecer a outros objetos. Por exemplo, mais uma vez em animais, um comportamento para um cachorro poderia ser andar(), latir(), rosnar() e tudo mais que fizer sentido para quem estiver modelando a classe.

1.1.3 Os Quatro Pilares da Orientação a Objetos

A POO se sustenta sobre quatro pilares teóricos fundamentais. A aplicação conjunta desses pilares permite a construção de softwares mais modulares, flexíveis, reutilizáveis e fáceis de manter.

1. Abstração

A abstração é o processo mental de isolar os aspectos essenciais de uma entidade do mundo real ou do domínio do problema, ignorando os detalhes irrelevantes para o contexto do sistema.

No desenvolvimento de software, não é viável nem necessário representar uma entidade em sua totalidade física ou biológica. A abstração estabelece o **fronteira do contexto**:

- Em um sistema acadêmico, a abstração de uma entidade "Pessoa"focará em atributos como matrícula, histórico de notas e curso.

- Em um sistema hospitalar, a mesma entidade "Pessoa"será abstraída focando em tipo sanguíneo, histórico médico e alergias.

A abstração correta garante que as classes contenham apenas o que é estritamente necessário para que o sistema cumpra seu propósito.

2. Encapsulamento

O encapsulamento é o mecanismo que agrupa dados (atributos) e procedimentos (métodos) em uma única unidade (o objeto) e **restringe o acesso direto** ao estado interno do objeto a partir do ambiente externo.

Os objetivos centrais do encapsulamento são:

- **Proteção de Dados**: Evitar que o estado interno do objeto seja alterado para valores inválidos ou incoerentes por agentes externos.

- **Ocultação da Implementação**: Permitir que a lógica interna de um método seja alterada pelo desenvolvedor sem impactar o restante do sistema, desde que a forma de interagir com ele continue a mesma.

O encapsulamento é operacionalizado por meio de **modificadores de acesso** (ou níveis de visibilidade):

- *Privado (Private)*: Os membros só podem ser acessados ou alterados por métodos da própria classe.
- *Público (Public)*: Os membros compõem a *interface pública* do objeto e podem ser acessados por qualquer outra parte do sistema.
- *Protegido (Protected)*: Os membros são acessíveis pela própria classe e por suas subclasses derivadas.

A comunicação segura com um objeto encapsulado ocorre mediante uma interface pública controlada, composta por métodos de validação, consulta e alteração.

3. Herança

A herança é um mecanismo de associação hierárquica que permite criar novas classes a partir de classes já existentes. Ela estabelece uma relação semântica do tipo **"é um"** ou **"é uma espécie de"**.

Nessa estrutura:

- **Classe Base (Superclasse ou Classe Mãe)**: Define as características e comportamentos genéricos compartilhados por uma família de entidades.
- **Classe Derivada (Subclasse ou Classe Filha)**: Herda automaticamente a estrutura e o comportamento da classe base, podendo adicionar novos atributos/métodos específicos ou redefinir comportamentos herdados.

O principal benefício da herança é a **reutilização de código** e a organização hierárquica do conhecimento no sistema. Quando um comportamento comum a várias classes precisa ser alterado, a modificação pode ser feita centralizadamente na classe base.

4. Polimorfismo

Originado do grego "muitas formas", o polimorfismo é a capacidade de objetos de diferentes classes derivadas responderem à mesma mensagem (chamada de método) de maneiras distintas e adequadas à sua especificidade.

Para que o polimorfismo ocorra, é necessário haver uma interface comum (proporcionada por herança ou por contratos abstratos). O sistema pode tratar um grupo heterogêneo de objetos derivados como se fossem do tipo genérico da classe base, mas, no momento da execução, o comportamento individual de cada um será acionado.

Existem duas formas principais de manifestação conceitual:

- **Sobrescrita (Overriding)**: A subclasse reescreve a implementação de um método herdado da classe base para adaptá-lo ao seu comportamento específico. A decisão de qual método executar ocorre dinamicamente em tempo de execução (*late binding*).
- **Sobrecarga (Overloading)**: Uma mesma classe possui métodos com o mesmo nome, porém com assinaturas (parâmetros e tipos) diferentes. A escolha da execução é resolvida em tempo de compilação.

O polimorfismo reduz o acoplamento do código, pois permite escrever rotinas genéricas que funcionam com novos tipos de objetos sem a necessidade de modificar as estruturas condicionais do programa.

—

1.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em C++.

1.2.1 Declaração de Classe e Modificadores de Acesso

Uma classe é declarada com a palavra-chave `class`. O corpo da classe define os níveis de acesso aos seus membros:

```
class SerVivo {
protected:
    std::string nome; // Acess vel pela classe e por classes derivadas
   (filhas)
public:
    // Acess vel de qualquer ponto do programa
};
```

- `private` (Padrão): Membros são acessíveis apenas pelos métodos da própria classe.
- `protected`: Membros são inacessíveis externamente, mas acessíveis dentro da própria classe e por suas subclasses derivadas.
- `public`: Membros podem ser acessados diretamente por qualquer objeto ou função do sistema.

1.2.2 Construtores e Lista de Inicialização

O construtor possui o mesmo nome da classe e é executado automaticamente no momento da instanciação. A sintaxe de **lista de inicialização de membros** (*Member Initializer List*) atribui valores aos membros antes da execução do corpo do construtor:

```
// Sintaxe: Construtor(parametros) : atributo(parametro) {}
SerVivo(string n) : nome(n) {}
```

1.2.3 Herança Pública

A herança estabelece uma relação do tipo "É UM". Para herdar publicamente de uma classe base:

```
class Humano : public SerVivo {
public:
    // Repassa o par metro 'n' para o construtor da classe base SerVivo
    Humano(string n) : SerVivo(n) {}
};
```

1.2.4 Polimorfismo Dinâmico: `virtual` e `override`

- `virtual` (na classe base): Declara um método que pode ser sobrescrito nas classes derivadas. Habilita o *dynamic dispatch* (ligação tardia em tempo de execução).
- `override` (na classe derivada): Garante em tempo de compilação que o método está sobrescrevendo exatamente uma função virtual da classe base.

```cpp
// Na Classe Base:
virtual void apresentar() { ... }

// Na Classe Derivada:
void apresentar() override { ... }
```

1.2.5 Destrutor Virtual (`virtual ~Destrutor()`)

Sempre que uma classe possui métodos virtuais e é manipulada por meio de ponteiros da classe base, seu destrutor **deve ser virtual**. Isso garante que o destrutor correto da classe derivada seja invocado na desalocação de memória.

```cpp
virtual ~SerVivo() {}
```

1.2.6 Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória

O polimorfismo dinâmico exige o uso de ponteiros ou referências da classe base armazenando o endereço de instâncias das classes filhas alocadas dinamicamente via operador `new`.

```cpp
// Armazenamento gen rico via ponteiro da classe base
vector<SerVivo*> seres;

// Instancia  o din mica
seres.push_back(new Humano("Arthur"));

// Chamada polim rfica via ponteiro (operador ->)
for (SerVivo* s : seres) {
    s->apresentar();
}

// Libera  o obrigat ria da mem ria alocada dinamicamente
for (SerVivo* s : seres) {
    delete s;
}
```

—

1.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em C++:

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// CLASSE BASE (Abstra  o)
class SerVivo {
protected:
    string nome; // Encapsulamento protegido

public:
    // Construtor com Lista de Inicializa  o
    SerVivo(string n) : nome(n) {}

    // M todo Virtual para Polimorfismo
    virtual void apresentar() {
```

```
17          cout << "Eu sou um ser vivo chamado " << nome << "." << endl;
18      }
19
20      // Destrutor Virtual
21      virtual ~SerVivo() {}
22  };
23
24  // HERAN A: Humano herda de SerVivo
25  class Humano : public SerVivo {
26  public:
27      Humano(string n) : SerVivo(n) {}
28
29      // SOBRESCRITA (Override)
30      void apresentar() override {
31          cout << "Ol  , eu sou o humano " << nome << ", mestre das
    civiliza  es." << endl;
32      }
33  };
34
35  class Elfo : public SerVivo {
36  public:
37      Elfo(string n) : SerVivo(n) {}
38
39      void apresentar() override {
40          cout << "Sauda  es , eu sou o elfo " << nome << ", guardi o
    das florestas." << endl;
41      }
42  };
43
44  class Fada : public SerVivo {
45  public:
46      Fada(string n) : SerVivo(n) {}
47
48      void apresentar() override {
49          cout << "Encantado! Eu sou a fada " << nome << ", portadora da
    magia." << endl;
50      }
51  };
52
53  int main() {
54      // Vetor de ponteiros para a classe base (Polimorfismo)
55      vector<SerVivo*> seres;
56
57      // Aloca  o din mica de mem ria
58      seres.push_back(new Humano("Arthur"));
59      seres.push_back(new Elfo("Legolas"));
60      seres.push_back(new Fada("Luna"));
61      seres.push_back(new SerVivo("Pedro"));
62
63      cout << "=== Apresenta  es no mundo de fantasia ===" << endl;
64
65      // Execu  o polim rfica
66      for (SerVivo* s : seres) {
67          s->apresentar();
68      }
69
70      // Desaloca  o de mem ria (Good Practices)
71      for (SerVivo* s : seres) {
```

```
72          delete s;
73      }
74
75      return 0;
76  }
```

Listing 1.1: Exemplo de Aplicação de POO em C++

—

1.4 Exercícios Práticos do Módulo

Exercício 1: Abstração e Encapsulamento (Sistema de Combate de Robôs)

Crie uma classe `Robo` contendo os atributos `modelo` (string), `versao` (int), `potenciaLaser` (float) e `integridade` (int). Implemente o método `disparar(Robo &alvo)` que reduz a integridade do alvo com base na potência do laser do atacante. Instancie dois robôs e simule um confronto.

Exercício 2: Herança e Modificadores de Acesso (Persona RPG)

Implemente a classe base `Pessoa` com atributos privados `nome` e `idade`. Crie as classes derivadas `Protagonista` (com o atributo `nivel`) e `Personagem` (com o atributo `rank`). Demonstre o acesso correto aos membros através dos construtores da classe pai.

Exercício 3: Polimorfismo Dinâmico (Membros do Inatel)

Crie a classe base `MembroInatel` com o método virtual `identificar()`. Derive as classes `Coordenador` (atributo `departamento`) e `Pesquisador` (atributo `laboratorio`). Utilize ponteiros da classe base para chamar o método `identificar()` de forma polimórfica.

Exercício 4: Polimorfismo em Coleções (Conselho das Terras Ancestrais)

Crie a classe base `MembroConselho` e as derivadas `Anao`, `Orc` e `Draconato`. Armazene instâncias dessas classes em um `std::vector<MembroConselho*>` e percorra a coleção invocando o método sobrescrito `saudar()`.

Capítulo 2

Relatório 7: Programação Orientada a Objetos em C#

2.1 Fundamentos Teóricos e Pilares da POO

No Relatório 6, os pilares da POO foram apresentados em C++, uma linguagem que exige controle manual da memória (`new` e `delete`). O **C#** mantém os mesmos conceitos, mas automatiza a liberação de memória por meio do **Garbage Collector** (coletor de lixo) e oferece uma sintaxe mais enxuta, principalmente no uso de **propriedades** para o encapsulamento. Os quatro pilares continuam valendo, agora ilustrados com o inventário do herói Link, do universo de *The Legend of Zelda*:

- **Abstração**: Todo objeto do jogo é modelado apenas com o que importa para o sistema. Um `Item` precisa somente de um `Nome` e de uma forma de ser usado.
- **Encapsulamento**: O acesso aos dados é controlado por propriedades. O dano de uma `Arma` pode ser lido por qualquer parte do programa, mas só pode ser alterado pela própria classe.
- **Herança**: `Arma` e `Escudo` reaproveitam a estrutura de `Item` (relação "**É UM**": uma Arma é um Item).
- **Polimorfismo**: Uma lista do tipo `Item` guarda armas, escudos e itens comuns; ao chamar `Usar()`, cada objeto executa a sua própria versão do método.

Além dos pilares, este relatório introduz os **relacionamentos entre classes**, que descrevem como um objeto pode conter ou utilizar outros objetos (relação "**TEM UM**"):

- **Composição (relação forte)**: A parte não existe sem o todo. A parte é criada **dentro** do objeto dono e "morre" junto com ele. Exemplo: o inventário de Link nasce dentro do próprio herói.
- **Agregação (relação fraca)**: A parte existe de forma independente do todo. O objeto dono apenas **guarda uma referência** a um objeto criado fora dele. Exemplo: a Master Sword é forjada antes e só depois é adicionada ao inventário de Link.

2.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em C#. Os exercícios deste relatório devem ser resolvidos **apenas** com as estruturas documentadas aqui.

2.2.1 Estrutura do Programa e Saída no Console

Todo programa C# começa pelo método `Main`. As diretivas `using` importam os recursos da linguagem: `System` (para o `Console`) e `System.Collections.Generic` (para a `List`). O caractere `$` antes das aspas ativa a **interpolação de strings**, permitindo inserir valores entre chaves diretamente no texto.

```
using System;
using System.Collections.Generic;

public class Program
{
    public static void Main(string[] args)
    {
        string heroi = "Link";
        Console.WriteLine($"O herói se chama {heroi}."); // interpolação
        Console.WriteLine("\n=== Fim ===");               // \n = quebra
    de linha
    }
}
```

2.2.2 Declaração de Classe e Propriedades

Uma classe é declarada com a palavra-chave `class`. Em C#, os atributos costumam ser expostos como **propriedades**, que controlam a leitura (`get`) e a escrita (`set`) de cada dado:

```
public class Magia
{
    // Leitura e escrita livres, com valor inicial "Nenhum"
    public string Efeito { get; set; } = "Nenhum";
}

// Dentro da classe Arma:
// Leitura livre, mas somente a própria classe pode alterar o valor
public int DanoBase { get; private set; }

// Dentro da classe Link:
// Campo privado: invisível fora da classe (convenção: prefixo _)
private List<Item> _inventario;
```

- `{ get; set; }`: A propriedade pode ser lida e alterada de qualquer ponto do programa.
- `{ get; private set; }`: A propriedade pode ser lida de qualquer ponto, mas somente os métodos da própria classe podem alterá-la. Tentar alterá-la na `Main` gera **erro de compilação**.
- `= valor`: Define o valor inicial da propriedade no momento em que o objeto é criado.
- `private`: Membro acessível apenas pelos métodos da própria classe.

2.2.3 Construtores e a Palavra-chave `this`

O construtor possui o mesmo nome da classe, não tem tipo de retorno e é executado automaticamente quando o objeto é criado com o operador `new`. A palavra-chave `this` referencia o próprio objeto que está sendo construído:

```
public Item(string nome)
{
    this.Nome = nome;
    Console.WriteLine($"[Item] {Nome} foi criado.");
}

// Instanciação: o construtor é chamado automaticamente
Item rupee = new Item("Rupee Azul");
```

Diferentemente do C++, não é necessário liberar a memória com `delete` nem declarar destrutores: o Garbage Collector remove automaticamente os objetos que não são mais utilizados.

2.2.4 Herança e Repasse ao Construtor Base (`: base(...)`)

A herança é indicada por dois-pontos após o nome da classe filha. O construtor da filha repassa os parâmetros necessários ao construtor da classe pai com `: base(...)`:

```
public class Arma : Item   // Arma É UM Item
{
    public int DanoBase { get; private set; }

    // "nome" vai para o construtor de Item; "dano" fica na Arma
    public Arma(string nome, int dano) : base(nome)
    {
        this.DanoBase = dano;
    }
}
```

2.2.5 Polimorfismo: `virtual` e `override`

- `virtual` (na classe base): Declara que o método **pode** ser reescrito pelas classes filhas. Sem ele, o C# sempre executaria a versão da classe pai.
- `override` (na classe derivada): Indica que o método está, intencionalmente, reescrevendo um método `virtual` da classe base.

```
// Na Classe Base (Item):
public virtual void Usar() { ... }

// Na Classe Derivada (Arma):
public override void Usar() { ... }
```

2.2.6 Reaproveitando o Comportamento do Pai: `base.Metodo()`

Dentro de um método sobrescrito, `base.Metodo()` executa a versão da classe pai. Isso permite reaproveitar parte da lógica do pai e apenas **acrescentar** o comportamento específico da filha:

```
public override void Usar()
{
    base.Usar(); // executa primeiro o Usar() de Item
    Console.WriteLine($"É um Escudo de Defesa: {Defesa}. Link se defende
    !");
}
```

2.2.7 Composição e Agregação

A diferença entre os dois relacionamentos está em **onde a parte é criada**:

```
// COMPOSIÇÃO: a Magia é criada DENTRO do construtor da Arma
public Arma(string nome, int dano) : base(nome)
{
    this.DanoBase = dano;
    this.MagiaAssociada = new Magia();
}

// AGREGAÇÃO: o Item é criado FORA de Link e apenas recebido como parâ
   metro
public void AdicionarItem(Item item)
{
    this._inventario.Add(item);
}
```

2.2.8 Coleções Genéricas: `List<T>` e `foreach`

A `List<T>` é uma coleção de tamanho dinâmico, em que `T` é o tipo dos objetos armazenados. Uma lista do tipo da classe pai (`List<Item>`) aceita objetos de qualquer classe filha, o que possibilita o polimorfismo ao percorrê-la com `foreach`:

```
List<Item> inventario = new List<Item>(); // criação da lista vazia
inventario.Add(new Arma("Master Sword", 30)); // adiciona um objeto
Console.WriteLine($"Total: {inventario.Count}"); // quantidade de itens

// Cada objeto executa a SUA versão de Usar()
foreach (var item in inventario)
{
    item.Usar();
}
```

- `.Add(objeto)`: Insere um objeto no final da lista.
- `.Count`: Retorna a quantidade de elementos da lista.
- `foreach (var x in lista)`: Percorre a lista elemento por elemento; a palavra `var` deixa o compilador deduzir o tipo da variável.
- `if (a != b)`: Executa o bloco apenas se os valores forem diferentes.

2.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em C#:

```
using System;
using System.Collections.Generic;

// todo: CLASSE BASE
// molde principal, todo item do jogo tem um nome
// as classes filhas vão herdar daqui
public class Item
{
    // todo: ENCAPSULAMENTO
```

```
10      // { get; set; } é a forma do C# de expor o atributo de forma
    controlada
11      // get = pode ler, set = pode alterar
12      public string Nome { get; set; }
13
14      // todo: CONSTRUTOR
15      // roda automaticamente quando o objeto é criado
16      public Item(string nome)
17      {
18          this.Nome = nome;
19          Console.WriteLine($"[Item] {Nome} foi criado.");
20      }
21
22      // todo: VIRTUAL
23      // avisa que as classes filhas podem reescrever esse método
24      // sem isso o C# ia sempre chamar essa versão aqui, ignorando as
    filhas
25      public virtual void Usar()
26      {
27          Console.WriteLine($"\n--- {Nome} ---");
28          Console.WriteLine("Usando um item genérico.");
29      }
30  }
31
32  // todo: CLASSE SEPARADA (sem herança)
33  // Magia não é um Item, então não herda - é só uma classe independente
34  // vai ser usada dentro de Arma mais pra frente (isso é Composição)
35  public class Magia
36  {
37      public string Efeito { get; set; } = "Nenhum";
38
39      public void Ativar()
40      {
41          Console.WriteLine($"Mágica ativada: {Efeito}!");
42      }
43  }
44
45  // todo: HERANÇA
46  // Arma herda de Item, já nasce com o atributo Nome
47  // a relação é: "Arma É UM Item"
48  public class Arma : Item
49  {
50      // private set = só a própria classe pode alterar o DanoBase
51      public int DanoBase { get; private set; }
52
53      // todo: COMPOSIÇÃO
54      // Arma tem uma Magia dentro dela
55      // quando a Arma é criada, a Magia nasce junto com ela
56      public Magia MagiaAssociada { get; set; }
57
58      // repassa o nome pro construtor do pai com : base(nome)
59      public Arma(string nome, int dano) : base(nome)
60      {
61          this.DanoBase = dano;
62          this.MagiaAssociada = new Magia();
63      }
64
65      // todo: SOBRESCRITA (override)
```

```
66      // reescreve o Usar() com o comportamento específico da Arma
67      // o override avisa o compilador que isso é intencional
68      public override void Usar()
69      {
70          Console.WriteLine($"\n--- {Nome} ---");
71          Console.WriteLine($"É uma Arma de Dano: {DanoBase}");
72
73          if (MagiaAssociada.Efeito != "Nenhum")
74          {
75              MagiaAssociada.Ativar();
76          }
77      }
78  }
79
80  // todo: HERANÇA
81  // mesma ideia da Arma, só muda o atributo e o comportamento do Usar()
82  public class Escudo : Item
83  {
84      public int Defesa { get; private set; }
85
86      public Escudo(string nome, int defesa) : base(nome)
87      {
88          this.Defesa = defesa;
89      }
90
91      public override void Usar()
92      {
93          // todo: base.Usar()
94          // chama o Usar() do pai antes de adicionar o comportamento do
    Escudo
95          // útil quando você quer reaproveitar parte da lógica da classe
    pai
96          base.Usar();
97          Console.WriteLine($"É um Escudo de Defesa: {Defesa}. Link se
    defende!");
98      }
99  }
100
101 public class Link
102 {
103     public string Nome { get; set; } = "Link";
104
105     // todo: COMPOSIÇÃO
106     // a lista nasce junto com Link e morre junto com ele
107     // não faz sentido existir inventário sem um herói dono dele
108     private List<Item> _inventario;
109
110     public Link()
111     {
112         this._inventario = new List<Item>();
113         Console.WriteLine($"\n[Link] O herói {Nome} foi criado.");
114     }
115
116     // todo: AGREGAÇÃO
117     // os itens existem fora de Link, ele só guarda uma referência pra
    eles
118     // masterSword e hylianShield foram criados antes e vivem
    independente do herói
```

```
119     public void AdicionarItem(Item item)
120     {
121         this._inventario.Add(item);
122     }
123 
124     public void MostrarInventario()
125     {
126         Console.WriteLine($"\n{Nome} tem {_inventario.Count} itens no
    seu inventário:");
127 
128         // todo: POLIMORFISMO EM AÇÃO
129         // o foreach trata tudo como Item, mas cada objeto roda o seu pr
    óprio Usar()
130         foreach (var item in _inventario)
131         {
132             item.Usar();
133         }
134     }
135 }
136 
137 public class Program
138 {
139     public static void Main(string[] args)
140     {
141         Console.WriteLine("=== Iniciando Aventura em Hyrule ===");
142 
143         Arma masterSword = new Arma("Master Sword", 30);
144         masterSword.MagiaAssociada.Efeito = "Corte de Energia Sagrada";
145 
146         Escudo hylianShield = new Escudo("Hylian Shield", 50);
147 
148         Link linkHeroi = new Link();
149 
150         linkHeroi.AdicionarItem(masterSword);
151         linkHeroi.AdicionarItem(hylianShield);
152         linkHeroi.AdicionarItem(new Item("Rupee Azul"));
153 
154         linkHeroi.MostrarInventario();
155 
156         Console.WriteLine("\n=== Fim da Demonstração ===");
157     }
158 }
```

Listing 2.1: Exemplo de Aplicação de POO em C#

2.4 Exercícios Práticos do Módulo

Exercício 1: Encapsulamento e Construtores (Defesa de Minas Tirith)

Durante o cerco a Minas Tirith, cada combatente convocado deve ser registrado com seu nome, povo, posto e o número do Círculo da cidade (de 1 a 7) que irá defender. Crie a classe `CombatenteDeGondor`:

- Propriedades `Nome`, `Povo`, `Posto` (string) e `Circulo` (int), todas com `private set`, inicializadas por um construtor que imprime uma mensagem de convocação.
- Propriedade `Armamento` com `private set` e valor inicial "Desarmado", alterada somente pelo método `Equipar(string arma)`.
- Método `ApresentarUnidade()`, que exibe os dados do combatente e só exibe o armamento quando ele for diferente de "Desarmado".

Na `Main`, instancie Legolas (Elfo, Arqueiro, Círculo 1) equipado com o "Arco dos Galadhrim", Peregrin Took (Hobbit, Guarda da Cidadela, Círculo 7) sem armamento e um terceiro combatente à escolha. Chame `ApresentarUnidade()` em todos. Por fim, tente alterar `Posto` diretamente na `Main`, observe o erro de compilação e comente a linha.

Exercício 2: Herança e Polimorfismo (Batalha de Exibição Pokémon)

Crie a classe base `Pokemon` com as propriedades `Especie` (string) e `Nivel` (int, com `private set`), ambas recebidas no construtor, e o método virtual `EntrarEmCampo()`, que imprime a espécie, o nível e um ataque genérico (Investida). Derive as classes:

- `TipoPlanta`, com a propriedade `GolpeEspecial` (string). Sobrescreva `EntrarEmCampo()` **sem** chamar a versão do pai.
- `TipoEletrico`, com a propriedade `Voltagem` (int, com `private set`). Sobrescreva `EntrarEmCampo()` chamando `base.EntrarEmCampo()` antes de exibir a descarga elétrica.

Na `Main`, crie uma `List<Pokemon>` com um Sceptile (`TipoPlanta`), um Jolteon (`TipoEletrico`) e um Eevee (`Pokemon` genérico). Exiba a quantidade de Pokémon em campo com `Count` e percorra a lista com `foreach` chamando `EntrarEmCampo()`.

Exercício 3: Composição e Agregação (O Grimório de Frieren)

A elfa Frieren carrega um grimório que nasce com ela e registra os feitiços que ela aprende (**composição**), e viaja com companheiros que existem independentemente dela (**agregação**). Implemente:

- `Feitico`: propriedade `Nome` e método `Conjurar()`.
- `Grimorio`: lista privada de `Feitico` criada no construtor; método `Registrar(string nomeFeitico)`, que cria o `Feitico` **dentro** do grimório; e método `ListarFeiticos()`, que exibe a quantidade e conjura cada um.
- `Companheiro`: propriedades `Nome` e `Funcao`, e método `Apresentar()`.
- `Maga`: propriedade `Nome`, propriedade `Grimorio` (com `private set`) criada no construtor, lista privada de `Companheiro` e os métodos `RecrutarCompanheiro(Companheiro c)` e `MostrarGrupo()`.

Na `Main`, crie Fern (Maga Aprendiz) e Stark (Guerreiro) **antes** de Frieren, recrute ambos, registre três feitiços pelo grimório de Frieren, exiba o grupo e o grimório e, por fim, chame `Apresentar()` diretamente em Stark. Explique, em um comentário no código, qual relacionamento é composição e qual é agregação.

Exercício 4: Revisão Geral (Arquivos Proibidos da Miskatonic)

O bibliotecário Henry Armitage cataloga os relatos de entidades cósmicas guardados na Universidade Miskatonic. Implemente:

- `EntidadeCosmica` (classe base, **não** abstrata): propriedades `Nome` e `Origem` (valor inicial "Desconhecida"), construtor que imprime o registro da entidade e método virtual `Manifestar()`, que só exibe a origem quando ela for diferente de "Desconhecida".
- `Profundo`: propriedade `Profundidade` (int, com `private set`); sobrescreve `Manifestar()` **sem** chamar o pai.
- `MiGo`: propriedade `Artefato` (string); sobrescreve `Manifestar()` chamando `base.Manifestar()` antes do seu comportamento.
- `Pesquisador`: propriedade `Nome`, lista privada de `EntidadeCosmica` criada no construtor (composição), método `Catalogar(EntidadeCosmica e)` (agregação) e método `LerCatalogo()` (polimorfismo com `foreach`).

Na `Main`, crie um `Profundo`, um `MiGo` com `Origem` igual a "Yuggoth" e uma `EntidadeCosmica` genérica ("A Cor que Caiu do Espaço"); catalogue as três no pesquisador Henry Armitage e chame `LerCatalogo()`.

Capítulo 3

Relatório 8: Programação Orientada a Objetos em Python

3.1 Fundamentos Teóricos e Pilares da POO

Nos Relatórios 6 e 7, os pilares da POO foram apresentados em C++ e em C#, linguagens que usam palavras-chave para controlar o acesso aos dados (`private`, `public`) e para permitir a sobrescrita de métodos (`virtual`, `override`). O **Python** mantém os mesmos conceitos, mas troca boa parte dessas palavras-chave por **convenções**: não existe `new`, o construtor sempre se chama `__init__`, o próprio objeto é recebido de forma explícita pelo parâmetro `self` e o nível de acesso de um atributo é indicado pelo seu nome. Assim como no C#, a memória é liberada automaticamente pelo **Garbage Collector**.

A tabela abaixo relaciona as estruturas já vistas em C# com as equivalentes em Python:

Conceito	**C# (Relatório 7)**	**Python (Relatório 8)**
Construtor	`public Item(string nome)`	`def __init__(self, nome)`
Próprio objeto	`this`	`self`
Instanciação	`new Item(nome)`	`Item(nome)`
Atributo privado	`private`	prefixo `__` (convenção)
Herança	`class Arma : Item`	`class Arma(Item)`
Construtor do pai	`: base(nome)`	`super().__init__(nome)`
Método do pai	`base.Usar()`	`super().usar()`
Sobrescrita	`virtual` + `override`	basta redefinir o método
Coleção	`List<Item>` e `.Add()`	`list[Item]` e `.append()`
Repetição	`foreach (var x in lista)`	`for x in lista:`

Os quatro pilares continuam valendo, agora ilustrados com um mini-RPG em que os heróis Arthur e Merlin se enfrentam:

- **Abstração**: Cada objeto guarda apenas o que importa para a batalha. Um `Personagem` precisa de nome, vida, nível, uma arma e um inventário.
- **Encapsulamento**: O dano de uma `Arma` é privado e só pode ser lido e alterado pelos métodos `get_dano()` e `set_dano()`, que impedem valores inválidos.
- **Herança**: `Guerreiro` e `Mago` reaproveitam a estrutura de `Personagem` (relação "**É UM**": um Guerreiro é um Personagem).
- **Polimorfismo**: Uma lista guarda guerreiros e magos; ao chamar `atacar()`, cada objeto executa a sua própria versão do método.

3.1.1 Interfaces: o Contrato entre Classes

A novidade deste relatório são as **interfaces**. Uma interface define **quais** métodos uma classe é obrigada a ter, sem dizer **como** eles funcionam. Ela funciona como um contrato: toda classe que "assina" a interface precisa implementar todos os seus métodos. No mini-RPG, a interface `IAcao` exige os métodos `atacar()` e `defender()`; por isso, `Guerreiro` e `Mago` são obrigados a implementar os dois, cada um do seu jeito. As interfaces:

- Garantem que classes diferentes, mas com comportamentos semelhantes, sigam o mesmo contrato.
- Facilitam a substituição de uma implementação por outra.
- Tornam o código mais modular e escalável.
- Permitem o uso do polimorfismo de forma limpa.

O Python não possui uma palavra-chave `interface`. Elas são representadas por **classes abstratas**, criadas com o módulo `abc`.

3.1.2 Relacionamentos entre Classes

A composição e a agregação, vistas no Relatório 7, continuam valendo (relação "**TEM UM**"):

- **Composição (relação forte)**: A parte é criada **dentro** do objeto dono e "morre" junto com ele. Exemplo: o inventário nasce dentro do próprio personagem.
- **Agregação (relação fraca)**: A parte é criada **fora** e o dono apenas guarda uma referência a ela. Exemplo: a Espada Longa e a Poção de Vida são criadas antes e só depois são entregues ao personagem.

3.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em Python. Os exercícios deste relatório devem ser resolvidos **apenas** com as estruturas documentadas aqui.

3.2.1 Estrutura do Programa e Saída no Console

O Python não usa chaves nem ponto e vírgula: os blocos são definidos pela **indentação** (4 espaços) após os dois-pontos. Comentários começam com `#`. O bloco `if __name__ == ...` marca o início do programa principal (o equivalente à `Main`). A letra `f` antes das aspas cria uma **f-string**, que permite inserir valores entre chaves diretamente no texto.

```
# comentários começam com #
if __name__ == "__main__":
    heroi = "Arthur"
    print(f"O herói se chama {heroi}.")  # f-string: interpolação
    print("\n--- Fim ---")                # \n = quebra de linha
```

3.2.2 Declaração de Classe, `__init__` e `self`

Uma classe é declarada com a palavra-chave `class`. O construtor é o método especial `__init__`, executado automaticamente quando o objeto é criado. Todo método recebe, como primeiro parâmetro, o `self`, que referencia o próprio objeto (o equivalente ao `this` do C#):

```
class Item:

    # roda automaticamente quando o objeto é criado
    def __init__(self, nome: str):
        self.nome = nome  # cria o atributo nome no próprio objeto

# Instanciação: em Python não existe "new"
pocao = Item("Poção de Vida")
print(pocao.nome)
```

- `def`: Declara um método (ou uma função). O corpo fica indentado abaixo dele.
- `self`: Na chamada, o Python preenche o `self` sozinho. Escrevemos `espada.usar()`, e não `espada.usar(espada)`.
- `nome: str`: **Anotação de tipo**. Documenta o tipo esperado do parâmetro, mas o Python não impede que outro tipo seja passado.
- `return`: Devolve um valor para quem chamou o método.

3.2.3 Encapsulamento por Convenção: `_` e `__`

O Python não tem as palavras `public`, `protected` e `private`. O nível de acesso é indicado pelo **prefixo do nome** do atributo:

```
class Personagem:

    def __init__(self, nome: str, vida: int, arma: Arma):
        self.nome = nome     # público: sem prefixo
        self._vida = vida    # protegido: um underline
        self.__nivel = 1     # privado: dois underlines
```

- **Sem prefixo** (`nome`): Atributo público, acessível de qualquer ponto do programa.
- **Um underline** (`_vida`): Atributo protegido. O Python **não** bloqueia o acesso; o underline é apenas um aviso de que o atributo é de uso interno da classe e das suas filhas.
- **Dois underlines** (`__nivel`): Atributo privado. O Python "embaralha" o nome do atributo, e ele deixa de ser encontrado fora da classe.

```
arthur = Guerreiro("Arthur", 100, espada)
print(arthur.nome)          # funciona: atributo público
print(arthur._vida)         # funciona, mas não é recomendado
print(arthur.__nivel)       # AttributeError: atributo não encontrado
print(arthur.get_nivel())   # forma correta: usando o getter
```

O embaralhamento também vale para as classes filhas: dentro de uma filha, `self.__nivel` não é encontrado. Para usar um atributo privado do pai, a filha deve chamar os métodos do próprio pai (como o getter).

3.2.4 Getters e Setters

Como um atributo privado não pode ser acessado de fora, a classe oferece métodos para lê-lo (**getter**) e para alterá-lo (**setter**). O setter valida o novo valor antes de aceitá-lo; se o valor for inválido, o anterior é mantido:

```
def get_dano(self):
    return self.__dano

def set_dano(self, novo_dano):
    if novo_dano > 0:
        self.__dano = novo_dano
    else:
        print("O dano deve ser positivo!")
```

3.2.5 Herança e `super().__init__()`

A classe pai é indicada entre parênteses, após o nome da classe filha. Se a filha **não** define um `__init__`, ela herda automaticamente o do pai:

```
class Guerreiro(Personagem):  # Guerreiro É UM Personagem
    ...

# usa o __init__ herdado de Personagem
arthur = Guerreiro("Arthur", 100, espada)
```

Quando a filha precisa de um atributo exclusivo, ela define o seu próprio `__init__` e repassa ao pai os parâmetros herdados com `super().__init__(...)` (o equivalente ao `base(...)` do C#). O exemplo abaixo é complementar e não faz parte do código de referência:

```
class Arqueiro(Personagem):

    def __init__(self, nome: str, vida: int, arma: Arma, flechas: int):
        # nome, vida e arma vão para o __init__ de Personagem
        super().__init__(nome, vida, arma)
        self.flechas = flechas  # atributo exclusivo do Arqueiro
```

3.2.6 Sobrescrita e `super().metodo()`

Em Python não existem `virtual` nem `override`: qualquer método pode ser sobrescrito. Basta a classe filha definir um método com o **mesmo nome** do método do pai. Dentro dele, `super().metodo()` executa a versão do pai (o equivalente ao `base.Metodo()` do C#), permitindo reaproveitar a sua lógica e apenas acrescentar o comportamento da filha:

```
# Na Classe Base (Personagem):
def mostrar_status(self):
    return f"Status: {self.nome} | Vida: {self._vida}"

# Na Classe Derivada (Arqueiro): reaproveita o texto do pai
def mostrar_status(self):
    return super().mostrar_status() + f" | Flechas: {self.flechas}"
```

3.2.7 Interfaces com `ABC` e `@abstractmethod`

Uma interface é criada como uma classe que herda de `ABC`. Os métodos obrigatórios recebem o decorador `@abstractmethod` e não possuem implementação:

```python
from abc import ABC, abstractmethod

class IAcao(ABC):  # herdar de ABC torna a classe abstrata

    @abstractmethod
    def atacar(self, alvo):
        pass  # sem implementação: só define o contrato

    @abstractmethod
    def defender(self):
        pass

# Mago É UM Personagem e "assina" o contrato de IAcao
class Mago(Personagem, IAcao):

    def atacar(self, alvo):
        print(f"<{self.nome}> lança uma BOLA DE FOGO em {alvo.nome}!")

    def defender(self):
        print(f"<{self.nome}> usa uma barreira arcana para defesa.")
```

- `from abc import ABC, abstractmethod`: Importa os recursos do módulo `abc` (*Abstract Base Classes*).
- `@abstractmethod`: Decorador que marca o método como obrigatório para as classes filhas. O prefixo `I` no nome da classe é uma convenção para indicar uma interface.
- `pass`: Bloco vazio. É usado quando a sintaxe exige um corpo, mas não há nada a executar.
- `class Mago(Personagem, IAcao)`: Uma classe pode herdar de mais de uma classe ao mesmo tempo, separando-as por vírgula.

Uma classe abstrata **não pode ser instanciada**. O mesmo erro acontece ao instanciar uma filha que esqueceu de implementar algum método abstrato:

```python
acao = IAcao()
# TypeError: Can't instantiate abstract class IAcao without an
# implementation for abstract methods 'atacar', 'defender'
```

3.2.8 Composição e Agregação

A diferença entre os dois relacionamentos está em **onde a parte é criada**:

```python
# COMPOSIÇÃO: o Inventario é criado DENTRO do __init__ do Personagem
self.inventario = Inventario()

# AGREGAÇÃO: o Item é criado FORA e apenas recebido como parâmetro
def adicionar_item(self, item: Item):
    self._itens.append(item)
```

3.2.9 Listas, `for` e `isinstance`

A `list` é a coleção de tamanho dinâmico do Python. Uma lista anotada com o tipo da classe pai aceita objetos de qualquer classe filha, o que possibilita o polimorfismo ao percorrê-la com `for`:

```python
itens = []                    # criação da lista vazia
itens.append(pocao)           # adiciona um objeto no final
print(f"Total: {len(itens)}")  # quantidade de elementos

# lista já preenchida, anotada com o tipo dos objetos
time_da_luta: list[IAcao] = [arthur, merlin]

for combatente in time_da_luta:  # percorre elemento por elemento
    if isinstance(combatente, Guerreiro):
        combatente.atacar(merlin)
    elif isinstance(combatente, Mago):
        combatente.atacar(arthur)
```

- `.append(objeto)`: Insere um objeto no final da lista.
- `len(lista)`: Retorna a quantidade de elementos da lista.
- `list[Classe]`: Anotação que indica o tipo dos objetos guardados na lista.
- `for x in lista:`: Percorre a lista elemento por elemento.
- `isinstance(objeto, Classe)`: Retorna `True` se o objeto for da classe informada (ou de uma filha dela). É útil para chamar um método que só existe em uma das classes filhas.
- `if`, `elif` e `else`: Estruturas de decisão. Condições podem ser combinadas com `and`, `or` e `not`, e comparações podem ser encadeadas, como em `0 <= x <= 10`.

3.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em Python:

```python
from abc import ABC, abstractmethod


# INTERFACE (contrato via classe abstrata)
# Python não tem interface nativa como C# ou Java
# ABC é a forma do Python simular isso
# toda classe que herdar de IAcao É OBRIGADA a ter atacar() e defender()
class IAcao(ABC):

    @abstractmethod
    def atacar(self, alvo):
        pass

    @abstractmethod
    def defender(self):
        pass


# CLASSE SIMPLES
# Arma não herda de nada, é só um molde com nome e dano
# é criada fora e entregue ao Personagem (isso é Agregação)
class Arma:

    def __init__(self, nome: str, dano: int):
        # ENCAPSULAMENTO - níveis de acesso em Python
        # Python não bloqueia o acesso de verdade, é uma convenção
```

```
27          # _nome  = um underline = "protegido", evitar mexer fora da
    classe
28          # __dano = dois underlines = "privado", o Python embaralha o
    nome
29          self._nome = nome
30          self.__dano = dano
31
32      # self indica que o método recebe o próprio objeto desta classe
33      def usar(self):
34          return f"{self._nome} (Dano: {self.__dano})"
35
36      # GETTER e SETTER
37      # como __dano é privado, criamos métodos pra ler e alterar ele
38      # isso é encapsulamento na prática: o objeto controla os seus dados
39      def get_dano(self):
40          return self.__dano
41
42      def set_dano(self, novo_dano):
43          if novo_dano > 0:
44              self.__dano = novo_dano
45          else:
46              print("O dano deve ser positivo!")
47
48
49  # AGREGAÇÃO
50  # Item existe por conta própria, independente de qualquer Inventario
51  # se o Inventario sumir, o Item continua existindo
52  class Item:
53
54      # __init__ é o construtor que usamos nas outras linguagens
55      def __init__(self, nome: str):
56          self.nome = nome
57
58
59  # AGREGAÇÃO (cont.)
60  # Inventario guarda referências pros Itens, mas não é dono deles
61  class Inventario:
62
63      def __init__(self):
64          self._itens = []
65
66      def adicionar_item(self, item: Item):
67          self._itens.append(item)
68          print(f"-> {item.nome} adicionado ao inventário.")
69
70      def listar_itens(self):
71          if not self._itens:
72              return "Inventário vazio."
73          # junta o nome de todos os itens em um texto, separados por ví
    rgula
74          return ", ".join([item.nome for item in self._itens])
75
76
77  # CLASSE BASE
78  # molde principal de todo personagem do jogo
79  # demonstra Encapsulamento e vai ser herdada por Guerreiro e Mago
80  class Personagem:
81
```

```
    def __init__(self, nome: str, vida: int, arma: Arma):
        self.nome = nome          # público - qualquer um pode acessar
        self._vida = vida         # protegido - evitar mexer fora da classe
        self.__nivel = 1          # privado - só a classe acessa diretamente
        # agregação: a Arma é criada fora e apenas recebida
        self.arma = arma
        # composição: o Inventario nasce e morre junto com o Personagem
        self.inventario = Inventario()

    def mostrar_status(self):
        return (f"Status: {self.nome} | Vida: {self._vida} | "
                f"Nível: {self.__nivel} | Arma: {self.arma._nome} | "
                f"Itens: {self.inventario.listar_itens()}")

    def receber_dano(self, dano):
        self._vida -= dano
        if self._vida < 0:
            self._vida = 0
        print(f"<{self.nome}> recebeu {dano} de dano. "
              f"Vida restante: {self._vida}")

    def subir_nivel(self):
        self.__nivel += 1
        print(f"{self.nome} subiu para o nível {self.__nivel}!")

    def get_nivel(self):
        return self.__nivel

    def set_vida(self, nova_vida):
        if nova_vida >= 0:
            self._vida = nova_vida
        else:
            print("A vida não pode ser negativa!")


# HERANÇA + INTERFACE
# Guerreiro herda de Personagem E implementa IAcao
# por isso é obrigado a ter atacar() e defender()
class Guerreiro(Personagem, IAcao):

    # SOBRESCRITA
    # implementa atacar() com o comportamento específico do Guerreiro
    def atacar(self, alvo):
        dano_total = self.arma.get_dano() * 1.5
        print(f"<{self.nome}> ATACA com força de GUERREIRO usando "
              f"{self.arma.usar()} em {alvo.nome}!")
        alvo.receber_dano(dano_total)

    def defender(self):
        print(f"<{self.nome}> se defende com o escudo, "
              f"reduzindo o próximo dano.")


# POLIMORFISMO
# Mago também implementa IAcao, mas o atacar() é completamente diferente
# mesma chamada, comportamento diferente: isso é polimorfismo
```

```python
class Mago(Personagem, IAcao):

    def atacar(self, alvo):
        dano_total = self.arma.get_dano() * 0.8 + 10
        print(f"<{self.nome}> lança uma BOLA DE FOGO (Mago) "
              f"em {alvo.nome}!")
        alvo.receber_dano(dano_total)

    def defender(self):
        print(f"<{self.nome}> usa uma barreira arcana para defesa.")


if __name__ == "__main__":
    print("--- Inicializando o Mini-RPG ---")

    # agregação: as armas são criadas fora e entregues aos personagens
    espada = Arma("Espada Longa", 15)
    cajado = Arma("Cajado Élfico", 12)

    # testando getter e setter antes de criar os personagens
    cajado.set_dano(espada.get_dano())

    arthur = Guerreiro("Arthur", 100, espada)
    merlin = Mago("Merlin", 80, cajado)

    # agregação: itens existem fora e são só referenciados pelo inventá
    rio
    pocao = Item("Poção de Vida")
    mapa = Item("Mapa Antigo")

    arthur.inventario.adicionar_item(pocao)
    merlin.inventario.adicionar_item(mapa)

    print("\n--- Status Inicial ---")
    print(arthur.mostrar_status())
    print(merlin.mostrar_status())

    # POLIMORFISMO EM AÇÃO
    # a lista do tipo IAcao guarda Guerreiro e Mago
    # o mesmo .atacar() chama métodos diferentes dependendo do objeto
    time_da_luta: list[IAcao] = [arthur, merlin]

    print("\n--- Simulação da Luta ---")
    for combatente in time_da_luta:
        if isinstance(combatente, Guerreiro):
            combatente.atacar(merlin)
        elif isinstance(combatente, Mago):
            combatente.atacar(arthur)

    # ENCAPSULAMENTO NA PRÁTICA
    # forma correta: usando getter e setter
    print(f"\nNível atual (via getter): {arthur.get_nivel()}")
    arthur.arma.set_dano(25)
    print(f"Dano atualizado via setter: {arthur.arma.get_dano()}")
    arthur.subir_nivel()

    print("\n--- Status Final ---")
    print(arthur.mostrar_status())
```

```
195     print(merlin.mostrar_status())
```

Listing 3.1: Exemplo de Aplicação de POO em Python

3.4 Exercícios Práticos do Módulo

Exercício 1: Encapsulamento e Herança (A Fogueira de Firelink)

Em Dark Souls, todo morto-vivo carrega almas e Frascos de Estus, que se recarregam nas fogueiras e nunca passam de 10. Os clérigos, além disso, conhecem um milagre próprio. Implemente:

- `MortoVivo`: atributos `nome` (público), `_almas` (protegido) e `__estus` (privado), inicializados no `__init__`; métodos `get_estus()` e `set_estus(quantidade)`, que só aceita valores de 0 a 10 (fora dessa faixa, exibe "Quantidade de Estus inválida!" e mantém o valor anterior); e método `mostrar_status()`, que **retorna** (não imprime) um texto com o nome, as almas e o Estus.
- `Clerigo`: herda de `MortoVivo` e possui o atributo exclusivo `milagre`, inicializando a parte herdada com `super().__init__(...)`. Sobrescreve `mostrar_status()`, reaproveitando o texto do pai com `super().mostrar_status()` e acrescentando o milagre.

No programa principal, crie um `Clerigo` à sua escolha e exiba o seu status. Em seguida, tente definir o Estus como 15 e depois como 10, exibindo o status novamente. Por fim, tente imprimir o atributo `__estus` do clérigo diretamente e observe o erro. Depois, transforme essa linha em comentário (`#`) para o programa voltar a rodar.

Exercício 2: Herança e Polimorfismo (Suprema em Overwatch)

Em Overwatch, todo herói pode usar a sua suprema, mas o efeito depende da sua função. Os suportes, além disso, conseguem curar a equipe. Implemente:

- `HeroiOverwatch`: atributos `codinome` e `funcao`, recebidos no `__init__`, e método `usar_suprema()`, que exibe uma suprema genérica.
- `HeroiTanque`: herda de `HeroiOverwatch`, **sem** `__init__` próprio, e sobrescreve `usar_suprema()`.
- `HeroiSuporte`: herda de `HeroiOverwatch`, **sem** `__init__` próprio, sobrescreve `usar_suprema()` e possui o método exclusivo `curar_equipe()`.

No programa principal, crie uma lista anotada como `list[HeroiOverwatch]` com um herói de cada uma das três classes, incluindo um herói da classe base. Percorra a lista com `for` chamando `usar_suprema()` de cada herói e, usando `isinstance`, chame `curar_equipe()` apenas nos suportes.

Exercício 3: Composição e Agregação (Infiltração no Palácio)

Em Persona 5, um grupo de estudantes chamado Phantom Thieves invade **Palácios**: mundos criados a partir do coração de pessoas corruptas, cada um conhecido pelo nome do seu dono. O líder do grupo, de codinome Joker, luta com a sua **Persona**, uma criatura que nasce de dentro dele e só existe enquanto ele existir (composição). Ao longo da história, ele recruta **aliados**: colegas que já existiam antes de entrar no grupo e continuam existindo fora dele (agregação).

Glossário:

- **Arcano**: categoria da Persona, inspirada nas cartas de tarô. No código, é só um texto.
- **Codinome**: apelido que cada membro usa durante as invasões, diferente do seu nome real.

Implemente:

- `Persona`: atributos `nome` e `arcano` e método `invocar()`, que exibe os dois dados.
- `Aliado`: atributos `nome` (nome real) e `codinome`.
- `Lider`: representa o Joker. Possui o atributo `codinome`; uma `Persona` criada **dentro** do `__init__`, com nome e arcano à sua escolha; uma lista `_equipe` que começa vazia; o método `recrutar(aliado: Aliado)`, que adiciona o aliado à `_equipe`; e o método `infiltrar(palacio)`, que recebe o nome do dono do Palácio, exibe o Palácio invadido, invoca a Persona e lista a equipe.

No programa principal, crie dois aliados **antes** do líder. Depois, crie o líder, recrute os dois e chame `infiltrar(...)`. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

Exercício 4: Revisão Geral (Ataque em Night City)

Em Cyberpunk, Night City é uma metrópole futurista controlada por grandes corporações. Para atacá-las, hackers se organizam em **células**, grupos que coordenam diferentes **unidades de rede** em um mesmo ataque. Toda unidade segue o mesmo protocolo: precisa saber executar uma invasão. Mas cada tipo de unidade faz isso do seu jeito. O **netrunner** usa o seu **cyberdeck** para quebrar o **gelo (ICE)** de um servidor. Já o **drone de vigilância** intercepta o sinal da rede.

Glossário:

- **Netrunner**: hacker que se conecta diretamente à rede para invadir sistemas.
- **Cyberdeck**: equipamento que o netrunner usa para hackear. Ele é implantado no corpo do netrunner, por isso não existe sem ele (composição).
- **Gelo (ICE)**: barreira de segurança de um servidor, parecida com um firewall.
- **Célula hacker**: grupo que reúne unidades que já existiam antes de entrar nele (agregação).

Implemente:

- `IUnidadeDeRede` (interface): classe abstrata que herda de `ABC`, com o método abstrato `executar_invasao()`.
- `Cyberdeck` (parte composta): atributo `modelo`.
- `OperadorNetrunner` (implementação com composição): implementa `IUnidadeDeRede`. O `__init__` recebe `nome` e o modelo do cyberdeck, e cria o `Cyberdeck` **dentro** dele. O método `executar_invasao()` exibe o nome do netrunner, o modelo do cyberdeck e que ele está quebrando o gelo (ICE) de um servidor.
- `DroneDeVigilancia` (outra implementação): também implementa `IUnidadeDeRede`. O `__init__` recebe `codigo`, a identificação do drone, e o método `executar_invasao()` exibe o código do drone e que ele está interceptando o sinal da rede.
- `CelulaHacker` (agregação): o `__init__` recebe `nome` e uma lista de membros já criados, anotada como `list[IUnidadeDeRede]`. O método `iniciar_ataque()` exibe o nome da célula e chama `executar_invasao()` de cada membro (polimorfismo).

No programa principal, crie um netrunner e um drone, junte os dois em uma célula e chame `iniciar_ataque()`. Por fim, tente criar um objeto `IUnidadeDeRede` diretamente e observe o erro. Depois, transforme essa linha em comentário (`#`) para o programa voltar a rodar.

Capítulo 4

Relatório 9: Programação Orientada a Objetos em Ruby

4.1 Fundamentos Teóricos e Pilares da POO

Nos Relatórios 6 a 8, os pilares da POO foram apresentados em C++, C# e Python. O Python mostrou que boa parte das palavras-chave (`private`, `virtual`, `override`) pode ser trocada por convenções. O **Ruby** vai além: nele, **tudo é objeto**, inclusive números, textos e as próprias classes, e a sintaxe foi desenhada para ser lida quase como uma frase em inglês. Os blocos não usam chaves nem dependem da indentação: eles terminam com a palavra `end`. Os objetos são criados com `.new`, e os atributos começam com `@` e **nunca** podem ser acessados diretamente de fora do objeto: a própria classe decide o que expor, criando métodos. Assim como no C# e no Python, a memória é liberada automaticamente pelo **Garbage Collector**.

A tabela abaixo relaciona as estruturas já vistas em Python com as equivalentes em Ruby:

Conceito	**Python (Relatório 8)**	**Ruby (Relatório 9)**
Construtor	`def __init__(self, nome)`	`def initialize(nome)`
Atributo do objeto	`self.nome`	`@nome`
Instanciação	`Item(nome)`	`Item.new(nome)`
Atributo privado	prefixo `__` (convenção)	toda variável `@`
Getter simples	`def get_nome(self)`	`attr_reader :nome`
Setter	`arma.set_dano(20)`	`arma.dano = 20`
Herança	`class Arma(Item)`	`class Arma < Item`
Construtor do pai	`super().__init__(nome)`	`super(nome)`
Método do pai	`super().usar()`	`super`
Interface	`ABC` + `@abstractmethod`	`module` + `include`
Coleção	`list` e `.append()`	`Array` e `<<`
Repetição	`for x in lista:`	`lista.each do |x| ... end`
Fim de bloco	indentação	`end`

Os quatro pilares continuam valendo, agora ilustrados com o universo do anime *Gachiakuta*. Nele, a sociedade se divide entre quem vive nas alturas e quem é jogado no **Abismo**, um lugar cheio de lixo. Lá vivem os **Catadores**, que dão vida a objetos descartados, chamados **Utensílios**, e se organizam em grupos de **Zeladores**:

- **Abstração**: Cada objeto guarda apenas o que importa para a história. Um `Habitante` do Abismo precisa de nome, origem e força.
- **Encapsulamento**: A força fica no atributo interno `@_forca`, que só pode ser lido e alterado pelos métodos `forca` e `forca=`, e o setter recusa valores inválidos. Já o nome e a origem podem ser lidos de fora, mas não alterados.

- **Herança**: `Catador` reaproveita a estrutura de `Habitante` (relação "**É UM**": um Catador é um Habitante).
- **Polimorfismo**: O grupo de Zeladores percorre os seus membros chamando `atacar`, e cada objeto executa a sua própria versão do método.

4.1.1 Módulos: Contratos e Mixins

A novidade deste relatório é a forma como o Ruby trata as interfaces. Ele não tem a palavra-chave `interface`, como o C#, nem classes abstratas nativas, como o `ABC` do Python. Em seu lugar, usa **módulos**: blocos de métodos que qualquer classe pode **incluir** com `include`. Um módulo pode ter dois papéis:

- **Contrato**: o módulo declara um método que só lança um `NotImplementedError`, e a classe que inclui o módulo deve escrever a sua própria versão do método. No código de referência, `CapacidadeEspirito` exige que o `Catador` implemente `invocar_espirito`.
- **Mixin**: o módulo já traz o método pronto, e toda classe que o incluir ganha esse comportamento sem precisar reescrevê-lo. No código de referência, `CapacidadeCombate` dá o método `entrar_em_combate` ao `Catador`.

Uma classe pode incluir vários módulos ao mesmo tempo. Essa é a alternativa do Ruby à herança múltipla vista no Python: uma classe tem um único pai, mas pode somar os comportamentos de quantos módulos precisar.

Há uma diferença importante em relação ao Relatório 8. No Python, o `ABC` impede que um objeto seja criado se faltar algum método abstrato. O Ruby **não confere nada antecipadamente**: o objeto é criado normalmente, e o `NotImplementedError` só aparece no momento em que o método que falta é chamado. Por isso, em Ruby, o que importa é se o objeto **sabe responder** a um método, e não de qual classe ele é. Essa ideia é chamada de *duck typing* ("se anda como um pato e grasna como um pato, então é um pato"), e o método `respond_to?` permite fazer essa pergunta ao objeto.

4.1.2 Relacionamentos entre Classes

A composição e a agregação, vistas nos relatórios anteriores, continuam valendo (relação "**TEM UM**"):

- **Composição (relação forte)**: A parte é criada **dentro** do objeto dono e "morre" junto com ele. Exemplo: o Utensílio nasce dentro do `initialize` do próprio Catador.
- **Agregação (relação fraca)**: A parte é criada **fora** e o dono apenas guarda uma referência a ela. Exemplo: os Catadores são criados antes e só depois são entregues ao grupo de Zeladores, e continuam existindo se o grupo deixar de existir.

4.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em Ruby. Os exercícios deste relatório devem ser resolvidos **apenas** com as estruturas documentadas aqui.

4.2.1 Estrutura do Programa e Saída no Console

O Ruby não tem uma função `main`: o arquivo é executado de cima para baixo, e o código solto depois das classes funciona como programa principal. Comentários começam com `#`. Blocos (classes, métodos, `if`, laços) terminam com `end`. O `puts` imprime um valor e pula uma linha. Dentro de aspas duplas, `#{...}` insere um valor no texto (**interpolação**), como a f-string do Python.

```ruby
# comentários começam com #
heroi = "Rudo"
puts "O herói se chama #{heroi}."   # interpolação com #{}
puts "Origem: " + "Aoiro"           # + junta dois textos
puts "\n--- Fim ---"                # \n = quebra de linha
```

- **Variáveis**: Não têm tipo declarado. Basta atribuir um valor, como em `heroi = ...`.
- **Parênteses opcionais**: Em uma chamada de método, os parênteses podem ser omitidos. `puts x` e `puts(x)` fazem a mesma coisa.
- `+`: Entre dois textos, junta os dois em um texto só.

4.2.2 Declaração de Classe, `initialize` e Atributos com `@`

Uma classe é declarada com `class` e termina com `end`. O construtor é o método `initialize`, executado automaticamente quando o objeto é criado com `.new`. Os atributos do objeto são as variáveis que começam com `@` (**variáveis de instância**). Elas não precisam ser declaradas antes: basta atribuir um valor.

```ruby
class Utensilio

  # roda automaticamente quando o objeto é criado com .new
  def initialize(nome, tipo)
    @nome = nome   # @ indica um atributo do próprio objeto
    @tipo = tipo
  end

  # a última expressão do método é o seu valor de retorno
  def mostrar_detalhes
    "Utensilio #{@nome} do tipo #{@tipo}."
  end
end

luvas = Utensilio.new("Luvas", "Ofensiva")  # instanciação
puts luvas.mostrar_detalhes
```

- `def ... end`: Declara um método. Um método sem parâmetros dispensa os parênteses, tanto na declaração quanto na chamada (`luvas.mostrar_detalhes`).
- `initialize`: O construtor, equivalente ao `__init__` do Python. Diferente do Python, ele não recebe o `self` como parâmetro.
- `@nome`: Variável de instância. Cada objeto tem a sua própria cópia, que existe enquanto o objeto existir.
- `.new`: Cria o objeto e chama o `initialize` com os argumentos recebidos.
- **Retorno implícito**: O método retorna o valor da sua última expressão, sem precisar de `return`. A palavra `return` também existe e pode ser usada para sair antes do fim do método.

- `self`: Dentro de um método, referencia o próprio objeto (como o `self` do Python e o `this` do C#), mas não aparece nos parâmetros. A expressão `self.class` (ou `objeto.class`) retorna a classe do objeto, e dentro de um texto ela aparece como o nome da classe, como em `#{self.class}`.

4.2.3 Encapsulamento: `attr_reader`, `attr_writer` e `attr_accessor`

Em Ruby, toda variável de instância é **inacessível de fora do objeto**: não existe sintaxe para ler um atributo com `@` de fora da classe. O nível de acesso é decidido pelos **métodos** que a classe oferece. Para não ser preciso escrever um getter para cada atributo, o Ruby oferece atalhos que criam esses métodos automaticamente:

```ruby
class Habitante
  attr_reader :nome, :origem   # cria os métodos nome e origem

  def initialize(nome, origem, forca_inicial)
    @nome = nome
    @origem = origem
    @_forca = forca_inicial     # interno: não ganhou attr_reader
  end
end

barado = Habitante.new("Barado", "Superficie", 800)
puts barado.nome          # funciona: attr_reader criou o método nome
barado.nome = "Outro"     # NoMethodError: o método nome= não existe
```

- `attr_reader :nome`: Cria apenas o getter `nome`. De fora, o atributo pode ser lido, mas não alterado.
- `attr_writer :nome`: Cria apenas o setter `nome=`.
- `attr_accessor :nome`: Cria os dois. No código de referência, é usado no `utensilio` do `Catador`.
- `:nome`: Um **símbolo**, isto é, um nome fixo precedido de dois-pontos. Os símbolos são usados para se referir a métodos e atributos pelo nome.
- `@_forca`: O `_` não muda nada para o Ruby, porque toda variável com `@` já é inacessível de fora. Ele é só uma convenção, usada no código de referência, para destacar um atributo interno, que não ganhou `attr_reader` e só é acessado pelos métodos da própria classe.
- `NoMethodError`: Erro lançado quando se chama um método que o objeto não possui, como `nome=` em uma classe que só tem `attr_reader :nome`.

4.2.4 Getters e Setters Manuais

Quando o atributo precisa de validação, o getter e o setter são escritos à mão. O setter é um método cujo nome termina com `=`; por isso, ele é chamado com a mesma sintaxe de uma atribuição:

```ruby
def forca              # getter: devolve o atributo interno
  @_forca
end

def forca=(novo_valor) # setter: o nome termina com =
  if novo_valor.is_a?(Numeric) && novo_valor.positive?
    @_forca = novo_valor
```

```ruby
  end
end

rudo.forca = 1100       # chama o setter forca=
puts rudo.forca         # chama o getter forca
```

No código de referência, um valor inválido é simplesmente ignorado. Para avisar o usuário, basta acrescentar um `else`. O exemplo abaixo é complementar e não faz parte do código de referência:

```ruby
def forca=(novo_valor)
  if novo_valor > 0
    @_forca = novo_valor
  else
    puts "A força deve ser positiva!"
  end
end
```

- `if`, `elsif`, `else` e `end`: Estruturas de decisão.
- **Comparações**: `==`, `!=`, `>`, `<`, `>=` e `<=`. Condições podem ser combinadas com `&&` (e), `||` (ou) e `!` (não), como em `x >= 1 && x <= 10`.
- `is_a?(Classe)`: Retorna `true` se o objeto for da classe informada ou de uma filha dela (o equivalente ao `isinstance` do Python). `Numeric` é a classe de todos os números.
- `positive?`: Retorna `true` se o número for maior que zero. Por convenção, os métodos terminados em `?` respondem sim ou não (`true` ou `false`).
- **Setter dentro da própria classe**: Deve ser chamado como `self.forca = valor`. Sem o `self.`, o Ruby entende que `forca = valor` cria uma variável local, e o setter não é executado.

4.2.5 Herança e `super(...)` no `initialize`

A classe pai é indicada com `<` depois do nome da classe filha. Para inicializar a parte herdada, a filha chama `super(...)` dentro do seu `initialize`, repassando os parâmetros de que o pai precisa (o equivalente ao `super().__init__(...)` do Python e ao `: base(...)` do C#):

```ruby
class Catador < Habitante   # Catador É UM Habitante

  def initialize(nome, origem, forca_inicial, nome_utensilio,
                 tipo_utensilio)
    # nome, origem e forca_inicial vão para o initialize de Habitante
    super(nome, origem, forca_inicial)
    @utensilio = Utensilio.new(nome_utensilio, tipo_utensilio)
  end
end
```

Se a filha **não** define um `initialize`, ela herda automaticamente o do pai e é criada com os mesmos parâmetros. O exemplo abaixo é complementar e não faz parte do código de referência:

```ruby
class Andarilho < Habitante   # sem initialize próprio
  def atacar
    puts "#{nome} arremessa uma pedra!"
  end
end

# usa o initialize herdado de Habitante
andarilho = Andarilho.new("Andarilho", "Abismo", 500)
```

- `class Filha < Pai`: Declara a herança (relação "É UM").
- **O que é herdado**: A filha herda os métodos do pai, inclusive os criados por `attr_reader`. Por isso, o `Andarilho` pode usar `nome` diretamente.
- **Variáveis com `@`**: Pertencem ao objeto, e não à classe que as criou. Diferente do `__` do Python, a filha enxerga as variáveis criadas no `initialize` do pai.

4.2.6 Sobrescrita e `super` em Métodos

Em Ruby não existem `virtual` nem `override`: qualquer método pode ser sobrescrito. Basta a classe filha definir um método com o **mesmo nome** do método do pai:

```ruby
# Na Classe Base (Habitante):
def atacar
  puts "#{nome} desfere um golpe basico!"
end

# Na Classe Derivada (Catador):
def atacar
  puts "#{nome} ataca usando o utensilio #{@utensilio.nome}!"
  puts "  [Tipo: #{@utensilio.tipo}]"
end
```

Dentro do método sobrescrito, a palavra `super` executa a versão do pai (o equivalente ao `super().metodo()` do Python e ao `base.Metodo()` do C#). Se a versão do pai **retorna** um texto, a filha pode reaproveitá-lo e apenas acrescentar o que é dela. O exemplo abaixo é complementar e não faz parte do código de referência:

```ruby
# Na Classe Base (Habitante): retorna um texto
def apresentacao
  "#{nome}, vindo de #{origem}"
end

# Na Classe Derivada (Catador): reaproveita o texto do pai
def apresentacao
  super + " | Utensilio: #{@utensilio.nome}"
end
```

- `super` (sem parênteses): Chama o método de mesmo nome no pai, repassando os mesmos argumentos que o método atual recebeu.
- `super()`: Chama o método do pai sem nenhum argumento.
- `super(a, b)`: Chama o método do pai com os argumentos informados. É a forma usada no `initialize` do `Catador`.

4.2.7 Módulos como Contrato: `module`, `include` e `NotImplementedError`

Um módulo é declarado com `module` e incluído em uma classe com `include`. Para funcionar como contrato, o método do módulo apenas lança um `NotImplementedError`. A classe que inclui o módulo escreve a sua própria versão do método, que tem prioridade sobre a do módulo:

```ruby
module CapacidadeEspirito
  def invocar_espirito
    raise NotImplementedError,
          "#{self.class} deve implementar invocar_espirito"
  end
end
```

```ruby

class Catador < Habitante
  include CapacidadeEspirito   # "assina" o contrato

  def invocar_espirito         # cumpre o contrato
    puts "#{nome} invoca o espirito do utensilio! Poder: #{forca * 5}"
  end
end
```

- `module Nome ... end`: Declara o módulo. O nome começa com letra maiúscula, como o de uma classe, mas um módulo não pode ser instanciado com `.new`.
- `include Modulo`: Acrescenta os métodos do módulo à classe. Os objetos da classe passam a responder a esses métodos.
- `raise Erro, mensagem`: Lança um erro e interrompe o programa, exibindo a mensagem. A mensagem é opcional: `raise NotImplementedError` sozinho também funciona.
- `NotImplementedError`: Erro que indica um método ainda não implementado.
- **Prioridade**: Quando a classe e o módulo têm um método com o mesmo nome, vale a versão da classe. Por isso, o `invocar_espirito` do `Catador` substitui o do módulo.

O Ruby não verifica o contrato antes da hora. O mesmo `raise NotImplementedError` pode ser usado diretamente em um método de uma classe base, para indicar que as filhas devem sobrescrevê-lo, mas a classe base continua podendo ser instanciada. O exemplo abaixo é complementar e não faz parte do código de referência:

```ruby
class Criatura
  def rugir
    raise NotImplementedError, "Cada criatura ruge do seu jeito"
  end
end

mancha = Criatura.new   # funciona: o objeto é criado normalmente
mancha.rugir            # NotImplementedError: o erro só aparece aqui
```

4.2.8 Módulos com Implementação Pronta (Mixins)

Um módulo também pode trazer métodos completos. Toda classe que o incluir ganha esse comportamento pronto, e uma mesma classe pode incluir quantos módulos precisar:

```ruby
module CapacidadeCombate
  def entrar_em_combate
    puts "#{self.class} entra em postura de combate!"
  end
end

class Catador < Habitante
  include CapacidadeEspirito
  include CapacidadeCombate    # uma classe, vários módulos
end
```

Dentro de um método de módulo, `self` é o objeto que está executando o método, ou seja, um objeto da classe que incluiu o módulo. Por isso, o módulo pode chamar os métodos desse objeto, como um getter criado por `attr_reader`, desde que a classe que o incluir tenha esse método. O exemplo abaixo é complementar e não faz parte do código de referência:

```ruby
module Rastreavel
  def emitir_sinal
    # nome vem do attr_reader da classe que incluir o módulo
    puts "Sinal de #{nome} detectado no Abismo!"
  end
end
```

4.2.9 Composição e Agregação

A diferença entre os dois relacionamentos está em **onde a parte é criada**:

```ruby
# COMPOSIÇÃO: o Utensilio é criado DENTRO do initialize do Catador
@utensilio = Utensilio.new(nome_utensilio, tipo_utensilio)

# AGREGAÇÃO: os Catadores são criados FORA e só recebidos pelo grupo
grupo = GrupoDeZeladores.new([rudo, enjin])
```

4.2.10 Arrays: Criação, `<<` e `.each`

O `Array` é a coleção de tamanho dinâmico do Ruby, equivalente à `list` do Python. Como a tipagem é dinâmica, um mesmo array pode guardar objetos de classes diferentes, o que possibilita o polimorfismo ao percorrê-lo:

```ruby
catadores = []            # array vazio
catadores << rudo         # adiciona no final
catadores.push(enjin)     # mesma coisa que <<
puts catadores.length     # quantidade de elementos

catadores.each do |c|     # percorre elemento por elemento
  c.atacar
end

catadores.each { |c| c.atacar }   # a mesma coisa, em uma linha
```

- `[]`: Cria um array vazio. `[a, b]` cria um array já preenchido.
- `array << objeto` e `array.push(objeto)`: Inserem um objeto no final do array.
- `array.length` (ou `array.size`): Retorna a quantidade de elementos.
- `.each`: Executa o **bloco** uma vez para cada elemento. O bloco pode ser escrito entre `do` e `end` ou entre chaves, e `|c|` declara a variável que recebe o elemento da vez.
- `def initialize(catadores = [])`: Um parâmetro pode ter **valor padrão**, usado quando o argumento não é informado.
- `puts array`: Imprime cada elemento do array em uma linha.

4.2.11 Transformar e Filtrar: `.map`, `.select` e `respond_to?`

Além do `.each`, os arrays têm métodos que percorrem os elementos e devolvem um **novo** array, sem alterar o original. As duas primeiras linhas de código do exemplo abaixo são complementares e não fazem parte do código de referência:

```ruby
# .map: novo array com o resultado do bloco para cada elemento
nomes = catadores.map { |c| c.nome }

# .select: novo array só com os elementos aprovados pelo bloco
```

```ruby
fortes = catadores.select { |c| c.forca > 1000 }

# introspecção: seleciona só quem possui o método entrar_em_combate
combatentes = @catadores.select { |c| c.respond_to?(:entrar_em_combate) }
combatentes.each(&:entrar_em_combate)
```

- `.map`: Devolve um novo array, do mesmo tamanho do original, com o valor da última expressão do bloco para cada elemento.
- `.select`: Devolve um novo array só com os elementos para os quais o bloco resultou em `true`.
- `objeto.respond_to?(:metodo)`: Retorna `true` se o objeto possui o método informado, inclusive quando ele veio de um módulo. É a forma de perguntar ao objeto o que ele sabe fazer (**introspecção**).
- `each(&:metodo)`: Atalho para `each { |c| c.metodo }`, que chama o método informado em cada elemento.

4.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em Ruby:

```ruby
# ------- Contexto: Gachiakuta ---------
# No mundo de Gachiakuta, a sociedade é dividida entre quem vive nas
# alturas e quem é jogado no Abismo - um lugar repleto de lixo e
# criaturas chamadas Manchas.
# Os Catadores são habitantes do Abismo que aprenderam a dar vida a
# objetos descartados, chamados Utensílios, usando o poder do espírito.
# Organizados em grupos de Zeladores, eles partem em missões para
# sobreviver e recuperar o que foi perdido nas profundezas.


# MÓDULOS (funcionam como interfaces em outras linguagens)
# definem um comportamento que qualquer classe pode "incluir"
# a classe que incluir este módulo deve implementar invocar_espirito;
# o Ruby não confere isso antes: o erro só aparece quando o método
# for chamado em um objeto que não o implementou
module CapacidadeEspirito
  def invocar_espirito
    raise NotImplementedError,
          "#{self.class} deve implementar invocar_espirito"
  end
end

# MÓDULO DE COMBATE
# outro módulo, mas esse já traz a implementação pronta
# qualquer classe que incluir CapacidadeCombate ganha o
# entrar_em_combate() de graça
module CapacidadeCombate
  def entrar_em_combate
    puts "#{self.class} entra em postura de combate!"
  end
end

```

```ruby

# COMPOSIÇÃO
# Utensilio não existe sozinho fora do Catador
# ele nasce e morre junto com ele
class Utensilio
  attr_reader :nome, :tipo

  # Construtor
  def initialize(nome, tipo)
    @nome = nome
    @tipo = tipo
  end

  def mostrar_detalhes
    "Utensilio #{@nome} do tipo #{@tipo}."
  end
end


# CLASSE BASE
# molde principal de todo habitante do Abismo
# as classes filhas vão herdar daqui
class Habitante

  # ENCAPSULAMENTO
  # em Ruby, toda variável de instância (as que começam com @) já é
  # inacessível de fora da classe: só dá para lê-la ou alterá-la por
  # meio de métodos
  # attr_reader cria apenas o getter para nome e origem: de fora, eles
  # podem ser lidos, mas não alterados
  # o _ em @_forca não muda nada para o Ruby: é só uma convenção para
  # indicar um atributo interno, que não ganhou attr_reader
  # como precisamos de validação na força (precisa ser um número
  # positivo), criamos getter e setter manuais
  attr_reader :nome, :origem

  def initialize(nome, origem, forca_inicial)
    @nome = nome
    @origem = origem
    @_forca = forca_inicial
  end

  # GETTER e SETTER manual
  # como @_forca é interno, criamos métodos pra ler e alterar ele de
  # forma controlada
  def forca
    @_forca
  end

  def forca=(novo_valor)
    # o if aqui funciona igual ao de outras linguagens:
    # só atualiza @_forca se novo_valor for um número positivo
    if novo_valor.is_a?(Numeric) && novo_valor.positive?
      @_forca = novo_valor
    end
  end

  # VIRTUAL (equivalente)
```

```ruby
  # método que as classes filhas vão sobrescrever com o comportamento
  # específico delas
  def atacar
    puts "#{nome} desfere um golpe basico!"
  end
end


# HERANÇA + MÓDULOS
# Catador herda de Habitante e inclui os dois módulos
# por isso deve implementar invocar_espirito (contrato do módulo)
class Catador < Habitante
  include CapacidadeEspirito
  include CapacidadeCombate

  # attr_accessor é como ter attr_reader e attr_writer juntos
  # ou seja, pode ler E alterar o utensílio de fora da classe
  # usamos accessor aqui porque o Catador pode trocar de Utensílio
  attr_accessor :utensilio

  # SUPER
  # super chama o initialize do pai (Habitante) passando os dados que
  # ele precisa
  # depois inicializamos o que é específico do Catador
  def initialize(nome, origem, forca_inicial, nome_utensilio,
   tipo_utensilio)
    super(nome, origem, forca_inicial)

    # COMPOSIÇÃO em ação
    # Utensilio é criado aqui dentro, ele pertence ao Catador
    @utensilio = Utensilio.new(nome_utensilio, tipo_utensilio)
    puts "#{nome} surge com o #{@utensilio.nome}."
  end

  # SOBRESCRITA (override)
  # reescreve o atacar() com o comportamento específico do Catador
  def atacar
    puts "#{nome} ataca usando o utensilio #{@utensilio.nome}!"
    puts "  [Tipo: #{@utensilio.tipo}]"
  end

  # IMPLEMENTAÇÃO DO MÓDULO
  # aqui a classe cumpre o contrato do CapacidadeEspirito
  # sem isso, chamar invocar_espirito lançaria o NotImplementedError
  # definido no módulo
  def invocar_espirito
    puts "#{nome} invoca o espirito do utensilio! Poder: #{forca * 5}"
  end
end


# AGREGAÇÃO
# GrupoDeZeladores guarda referências pra Catadores que já existiam
# fora dele
# se o grupo sumir, os catadores continuam existindo
class GrupoDeZeladores
  attr_accessor :catadores

```

```ruby
  def initialize(catadores = [])
    @catadores = catadores
  end

  def iniciar_missao(missao)
    puts "\n--- Grupo iniciando missao: #{missao} ---"

    # POLIMORFISMO EM AÇÃO
    # o mesmo .atacar() para todos, mas cada um executa o seu próprio
    @catadores.each do |c|
      c.atacar
    end
  end

  def listar_combatentes
    puts "\n--- Zeladores prontos para combate ---"

    # INTROSPECÇÃO com .select
    # .select percorre o array e retorna só os elementos que passam na
    # condição do bloco
    # o bloco é o trecho entre {} - é como um mini-método anônimo
    # executado pra cada elemento
    # |c| é a variável temporária que representa o elemento atual da
    # iteração
    # as barras || são a sintaxe do Ruby pra declarar esse parâmetro do
    # bloco, igual um (c) em outras linguagens
    # respond_to?(:entrar_em_combate) verifica se o objeto c possui
    # aquele método disponível
    combatentes = @catadores.select { |c| c.respond_to?(:
   entrar_em_combate) }
    combatentes.each(&:entrar_em_combate)
  end
end


# --- MAIN ---

rudo   = Catador.new("Rudo",   "Aoiro",  1000, "Luvas",        "Ofensiva
   ")
enjin  = Catador.new("Enjin",  "Abismo", 1200, "Guarda-chuva", "
   Velocidade")
barado = Habitante.new("Barado", "Superficie", 800)

rudo.forca = 1100
puts "\nRudo agora tem #{rudo.forca} de forca."

puts "\n--- Habilidades ---"
rudo.invocar_espirito
rudo.entrar_em_combate
# barado.invocar_espirito <- daria NoMethodError: Habitante não inclui
# o módulo CapacidadeEspirito

puts "\nComposicao:"
puts rudo.utensilio.mostrar_detalhes

# agregação: catadores já existiam antes do grupo ser criado
grupo = GrupoDeZeladores.new([rudo, enjin])

```

```ruby
grupo.iniciar_missao("Recuperar utensilios perdidos no Abismo")
grupo.listar_combatentes
```

Listing 4.1: Exemplo de Aplicação de POO em Ruby

4.4 Exercícios Práticos do Módulo

Exercício 1: Encapsulamento e Herança (O Cardápio do Afterlife)

Em *Cyberpunk 2077*, o Afterlife é o bar mais famoso de Night City, uma metrópole futurista, e é frequentado por mercenários. O cardápio tem coquetéis comuns e drinks que homenageiam **lendas**. Todo drink tem um nome, que nunca muda, e um preço, que o bar pode reajustar dentro da tabela da casa.
Glossário:

- **Lenda**: mercenário famoso de Night City, vivo ou morto, homenageado com um drink. No código, é só um texto.
- **Eurodólar**: a moeda de Night City. No código, o preço é só um número.

Implemente:

- `CoquetelAfterlife`: o `initialize` recebe `nome` e `preco`. O `nome` pode ser lido de fora, mas não alterado (use `attr_reader`). O preço fica guardado em `@_preco`, sem `attr_reader`, e o `initialize` atribui o valor recebido diretamente a ele. A classe tem o getter manual `preco`; o setter manual `preco=(valor)`, que só aceita valores de 5 a 200 (fora dessa faixa, exibe "Preço inválido!" e mantém o valor anterior); e o método `ficha`, que **retorna** (não imprime) um texto com o nome e o preço do drink.
- `DrinkDeLenda`: herda de `CoquetelAfterlife` e possui o atributo exclusivo `lenda` (com `attr_reader`). O `initialize` recebe `nome`, `preco` e `lenda` e repassa os dois primeiros ao pai com `super(nome, preco)`. Sobrescreve `ficha`, reaproveitando o texto do pai com `super` e acrescentando a lenda homenageada.

No programa principal, crie um array com um `CoquetelAfterlife` e dois `DrinkDeLenda`, à sua escolha, e percorra-o com `.each`, imprimindo a `ficha` de cada drink. Depois, tente definir, com o setter, o preço de um dos drinks de lenda como 500 e depois como 80, e imprima a ficha dele de novo. Por fim, tente alterar o nome de um dos drinks diretamente (`objeto.nome = ...`) e observe o erro. Depois, transforme essa linha em comentário (`#`) para o programa voltar a rodar.

Exercício 2: Abstração, Polimorfismo e Arrays (A Orquestra da Gala)

Em *Your Lie in April* (*Shigatsu wa Kimi no Uso*), jovens músicos de música clássica se apresentam em competições e concertos de gala. Em uma apresentação, cada instrumentista tem um papel: o **solista** toca a melodia principal e o **acompanhador** toca a base, seguindo o ritmo do solista. Todos são conduzidos pelo **regente**.

Glossário:

- **Solista**: músico que toca a melodia principal da peça, em destaque.
- **Acompanhador**: músico que toca a base da peça, seguindo o ritmo do solista.
- **Regente**: quem conduz a orquestra durante a apresentação (o maestro).
- **Peça**: a obra musical apresentada. No código, é só um texto com o nome dela.

Implemente:

- `Instrumentista` (abstração): o `initialize` recebe `nome` e `instrumento` (os dois com `attr_reader`). O método `interpretar(peca)` tem como corpo apenas um `raise NotImplementedError`, com uma mensagem à sua escolha.
- `Solista`: herda de `Instrumentista`, **sem** `initialize` próprio (ela usa o do pai), e sobrescreve `interpretar(peca)` para imprimir o nome, o instrumento e a peça, dizendo que ele conduz a melodia principal.
- `Acompanhador`: também herda de `Instrumentista`, **sem** `initialize` próprio, e sobrescreve `interpretar(peca)` para imprimir o nome, o instrumento e a peça, dizendo que ele acompanha o ritmo do solista.
- `Regente`: o `initialize` recebe `nome` e cria o array `@orquestra` vazio. Possui o método `convocar(instrumentista)`, que adiciona o instrumentista ao `@orquestra` com `<<`; o método `reger(peca)`, que imprime o nome do regente e a peça e, com `.each`, chama `interpretar(peca)` de cada instrumentista; e o método `escalacao`, que usa `.map` para **retornar** um novo array de textos, um por instrumentista, com o nome da classe dele (use `.class`) e o seu nome.

No programa principal, crie um `Solista`, um `Acompanhador` e um `Regente`, à sua escolha. Convoque os dois instrumentistas, chame `reger(...)` com uma peça à sua escolha e imprima, com `.each`, cada texto do array retornado por `escalacao`. Por fim, crie um `Instrumentista` diretamente e, na linha seguinte, chame `interpretar(...)` nele. Observe que o objeto é criado, mas a chamada gera um erro. Depois, transforme a linha da chamada em comentário (`#`) para o programa voltar a rodar.

Exercício 3: Composição e Agregação (D-Mail para o Passado)

Em *Steins;Gate*, um grupo de amigos mantém o Laboratório de Gadgets do Futuro, um pequeno laboratório de invenções em Tóquio. A invenção mais importante do grupo é o **PhoneWave**, um micro-ondas ligado a um celular que, por acidente, envia mensagens de texto para o passado. O aparelho foi montado dentro do laboratório e só existe nele (composição). Os membros do laboratório já eram pessoas comuns antes de entrar no grupo e continuam existindo fora dele (agregação).

Glossário:

- **PhoneWave**: o "Microondas-Telefone", nome que o aparelho recebe no anime.
- **D-Mail**: mensagem de texto enviada para o passado pelo PhoneWave. No código, é só um texto.
- **Lab Mem**: abreviação de *Laboratory Member*. Cada membro do laboratório recebe um número de membro. No código, esse número é um inteiro.

Implemente:

- `PhoneWave`: método `enviar_dmail(mensagem)`, que imprime a mensagem e avisa que ela foi enviada para o passado.
- `MembroDoLab`: o `initialize` recebe `numero` (o número de Lab Mem) e `nome`, os dois com `attr_reader`.
- `LaboratorioDeGadgets`: o `initialize` não recebe parâmetros, cria um `PhoneWave` **dentro** dele, guardado em `@phonewave`, e cria o array `@membros` vazio. Possui o método `admitir(membro)`, que adiciona um membro já criado ao `@membros` e imprime o número e o nome dele; e o método `experimento(mensagem)`, que imprime, com `.each`, o número e o nome de cada membro presente e depois envia a mensagem pelo `enviar_dmail` do PhoneWave guardado em `@phonewave`.

No programa principal, crie dois membros **antes** do laboratório, à sua escolha. Depois, crie o laboratório, admita os dois e chame `experimento(...)` com uma mensagem à sua escolha. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

Exercício 4: Módulos e Introspecção (Depoimentos em Baker Street)

Sherlock Holmes é um detetive que mora na Baker Street, em Londres. Ao investigar um crime, ele ouve todos os envolvidos, e cada um presta **depoimento** do seu jeito: a **testemunha ocular** conta o que viu e o suspeito apresenta o seu **álibi**. No fim, Holmes entrega à **Scotland Yard** quem pode ser detido, e só os suspeitos podem.
Glossário:

- **Depoimento**: relato de uma pessoa sobre o crime. **Depoente** é quem presta o depoimento.
- **Testemunha ocular**: pessoa que viu o crime acontecer.
- **Álibi**: a justificativa do suspeito de que estava em outro lugar na hora do crime.
- **Scotland Yard**: a polícia de Londres.

Implemente:

- `Depoente` (módulo-contrato): método `depor`, cujo corpo é apenas um `raise NotImplementedError`.
- `Detivel` (módulo com implementação pronta): método `ser_detido`, que imprime o `nome` da pessoa e que ela foi levada pela Scotland Yard. O `nome` vem do `attr_reader` da classe que incluir o módulo.
- `Envolvido` (classe base): o `initialize` recebe `nome` (com `attr_reader`). Ela não inclui nenhum módulo.
- `TestemunhaOcular`: herda de `Envolvido` e inclui **apenas** `Depoente`. Implementa `depor` para imprimir o nome e o que a testemunha viu (texto à sua escolha).
- `SuspeitoDoCrime`: herda de `Envolvido` e inclui `Depoente` **e** `Detivel`. Implementa `depor` para imprimir o nome e o álibi do suspeito (texto à sua escolha).
- `Investigacao` (agregação): o `initialize` recebe `caso` (o nome do caso) e cria o array `@envolvidos` vazio. Possui o método `registrar(envolvido)`, que adiciona um envolvido já criado ao array; o método `ouvir_todos`, que imprime o nome do caso e chama `depor` de cada envolvido com `.each` (polimorfismo); e o método `acionar_scotland_yard`, que usa `.select` com `respond_to?(:ser_detido)` para separar quem pode ser detido e chama `ser_detido` de cada um deles.

No programa principal, crie uma testemunha ocular e dois suspeitos, à sua escolha, **antes** da investigação. Depois, crie a investigação com um caso à sua escolha, registre os três e chame `ouvir_todos` e `acionar_scotland_yard`.